Workflows in Oracle APEX

A practical guide to tasks, workflows and their APIs

Disclaimer. This guide is based on practical experience and publicly available sources. It is a simplified reference, not an exhaustive or authoritative one. Details may be inaccurate or outdated, and behaviour can vary with the APEX version, configuration or environment. Always verify against the official Oracle documentation and test in a non-production environment before relying on any of it. Written for APEX 23.2–24.2.

Contents

1 What Is a Workflow in APEX? 2

2 Tasks 2

2.1 What a Task Is . . . 2
2.2 Task Definitions . . . 2
2.3 Task Definition Actions . . . 2
2.4 Task Details Page . . . 3
2.5 Participants and Roles . . . 3
2.6 Task States . . . 4
2.7 Unified Task List . . . 5
2.8 Task Parameters . . . 5
2.9 Comments . . . 5
2.10 Task Visibility Pitfalls . . . 6

3 Working with the APIs 6

3.1 Useful Views . . . 6
3.2 Reading Tasks . . . 6
3.3 Completing a Task from Code . . . 6
3.4 Passing Data Between Tasks . . . 6

4 Workflows 7

4.1 How a Workflow Runs . . . 7
4.2 Workflow States . . . 7
4.3 Activity States . . . 8
4.4 Parameters and Variables . . . 8
4.5 Development and Active Versions . . . 9

5 Troubleshooting 9

5.1 Error ORA-20987 When a Task Is Created . . . 9
5.2 Rolling Back Changes After a Workflow Is Terminated . . . 9
5.3 The Automation Does Not Run . . . 10
5.4 A User Cannot See a Task . . . 10

6 References 10

1 What Is a Workflow in APEX?

A workflow in APEX is a structured process that automates business rules, approvals and sequences of tasks. It can combine automated steps (PL/SQL, REST calls, e-mails) with human steps (approvals and actions performed by users).

Every workflow instance is started for one business record, identified by its **detail primary key** (`detail_pk`). This key is passed in when the workflow starts, carried through every activity and task, and can be referenced at any point to retrieve the record's data.

2 Tasks

2.1 What a Task Is

Tasks are the *human* steps of a workflow: steps that require someone to act before the workflow can continue.

Example. When a purchase request is submitted, it is sent to a reviewer who must approve or reject it. This is an approval task; its outcome decides whether the request moves on to the next approver or is closed.

2.2 Task Definitions

Tasks are defined in **Shared Components** → **Task Definitions**. There are two types:

- **Approval task:** the user approves or rejects. The outcome is stored in a workflow variable, typically read by a *Switch* activity to send the workflow down the right branch.
- **Action task:** the user simply completes the task (marks it as done). There is no approve/reject outcome.

2.3 Task Definition Actions

Task definition actions run automatically when something happens to a task. They can send an e-mail, send a push notification, or execute PL/SQL code. Unlike workflow activities, they run *inside the task context* and are tied to task events:

Task events		
Create	Complete	Claim
Cancel	Release	Delegate
Update Comment	Update Priority	Update Parameter
Request Information	Submit Information	Before Expire, Expire

> **Task action or workflow activity?**
> Use a task definition action when you need task-specific data, for example an e-mail that includes task parameters, or an update based on what the user entered in the task. If no task data is needed, a normal workflow activity is usually simpler.

2.4 Task Details Page

Each task has a **Task Details page**, which APEX can generate for you. It shows the data of one task instance and can differ from what is shown in the Unified Task List.

- The bind variable `:APEX$TASK_PK` holds the detail primary key of the task and can be used to query the related business record.
- What is displayed (subject, details) comes from the task definition's *Action Source*.
- The page can be customised freely, and its actions can be implemented with the `APEX_HUMAN_TASK` package.

Example: show the order number in the task subtitle by querying the record behind the task.

```
select order_id
  from orders
 where order_id = :APEX$TASK_PK;
```

A task can also have a deadline (due date) after which it expires.

2.5 Participants and Roles

Task participants define who can act on a task. They are of type *Potential Owner* or *Business Administrator*, and the task appears in their Unified Task List.

- **Potential owner:** can claim an unassigned task and thereby become its actual owner. A task can have several potential owners.
- **Actual owner:** works on a claimed task and can approve, reject or complete it.
- **Initiator:** the user who started the task. Can change its priority and cancel it (if enabled).
- **Business administrator:** oversees all tasks of a task definition; can delegate them, change their priority and add potential owners (if enabled).

Action	Initiator	Potential owner	Actual owner	Business admin
Claim	–	Yes, if the task is unassigned	–	–
Complete (approve / reject)	–	Yes, from the task list, or by claiming it on the details page	Yes, from the task list or the details page	–
Delegate	–	–	Yes	Yes
Release	–	–	Yes	–
Cancel (if enabled)	Yes	–	–	–
Add potential owner (if enabled)	–	–	–	Yes
Add comment	Yes	Yes	Yes	Yes
Set priority	Yes	–	–	Yes

Table 1: Task actions by role.

Good to know

- A task with a single potential owner is assigned to that user directly; with several potential owners it stays unassigned until someone claims it.
- An actual owner can delegate a task to another user, typically someone with the same role.
- Every task and workflow instance is stored by APEX and can be queried through views (Section 3).
- All task actions are available on the Task Details page, and each one also exists as a procedure or function in `APEX_HUMAN_TASK`, so it can be called from automations or other code.

2.6 Task States

Figure 1 and Table 2 show the life cycle of a task.

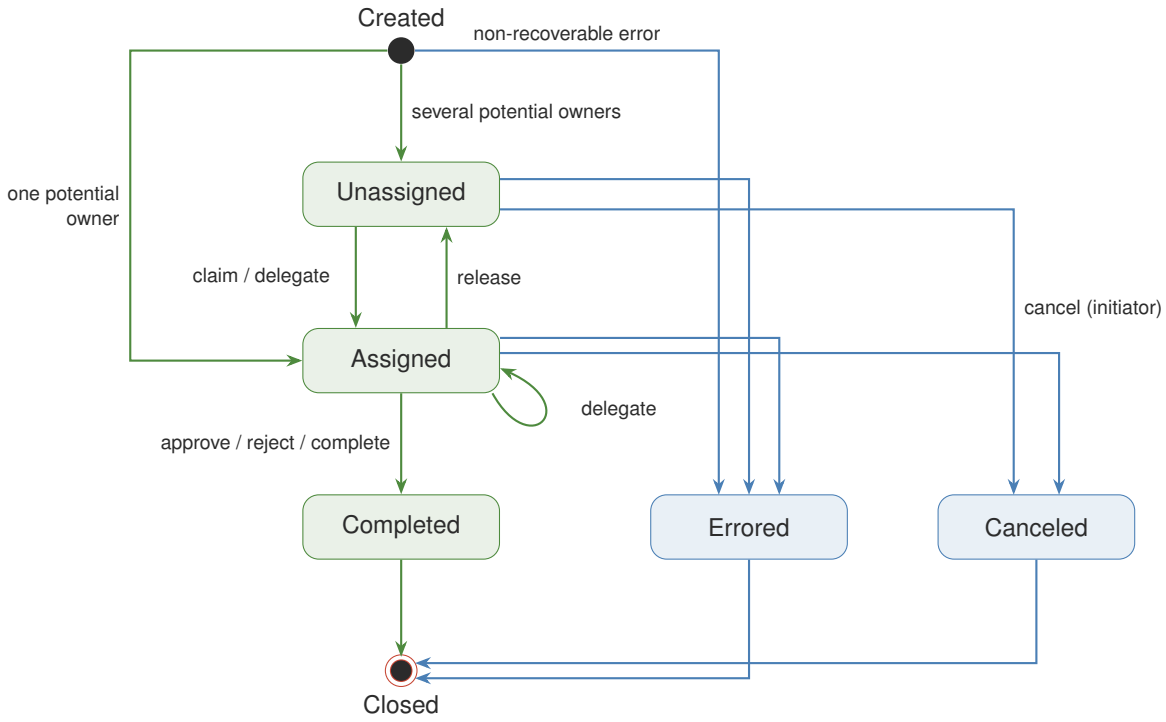


Figure 1: Task states and transitions. Green: normal flow; blue: exceptions.

State	Stage / category	Possible transitions
Created	Initial / normal	**Unassigned** if there are several potential owners; **Assigned** if there is one; **Errored** on a non-recoverable error.
Unassigned	Intermediate / normal	**Assigned** when a user claims it or it is delegated; **Errored**; **Canceled** by the initiator.
Assigned	Intermediate / normal	**Completed** when the owner approves, rejects or completes it; **Unassigned** when the owner releases it; **Assigned** (new owner) when delegated; **Errored**; **Canceled** by the initiator.
Completed	Final / normal	None.
Errored	Final / exception	None.
Canceled	Final / user interrupted	None.

Table 2: Task states and transitions.

2.7 Unified Task List

The Unified Task List works like an inbox for tasks. Its exact name and layout depend on your application, but it offers three report contexts:

- **My Tasks:** tasks where you are a potential or actual owner. You can approve, reject or complete them.
- **Admin Tasks:** tasks where you are business administrator. You can change their priority or delegate them.
- **Initiated by Me:** tasks you started. You can change their priority or cancel them.

Each task is shown with its subject and initiator. Clicking a task opens its Task Details page, which can be customised to show what the user needs. The generated page includes buttons for the standard actions; if some do not fit your process, re-implement them with the `APEX_HUMAN_TASK` package.

> **Deprecated package**
> `APEX_APPROVAL` is deprecated and will be removed in a future release. Use `APEX_HUMAN_TASK` instead; it offers the same procedures and functions.

2.8 Task Parameters

Task parameters are declared in the task definition. They are used to determine potential owners, to display information, or inside task action code. Their values are set when the workflow creates the task, and they can be updated later (for example from the Task Details page) and read back from another activity.

2.9 Comments

Comments are shared between everyone involved in a task. They are typically used to discuss a request before claiming it or to explain a delegation.

2.10 Task Visibility Pitfalls

- If an approval task does not allow the initiator to approve it, the task does not appear for the initiator, even if they are listed as a potential owner.
- Potential owners are evaluated when the task is created. A user given the required role *afterwards* will not see the existing task. Make sure roles are assigned before the workflow instance starts.
- If the workflow has already started, retrying the task (or canceling and recreating it) re-evaluates the potential owners and includes the new users.

3 Working with the APIs

3.1 Useful Views

View	Content
`APEX_WORKFLOWS`	All workflow instances, with their state and `detail_pk`.
`APEX_WORKFLOW_VARIABLES`	Current values of workflow variables.
`APEX_TASKS`	Human task instances (task ID, state, `detail_pk`, ...).
`APEX_TASK_PARAMETERS`	Parameter values of each task.

3.2 Reading Tasks

Display the details of one task, for example in a Classic Report on the Task Details page:

```
select *
  from table ( apex_human_task.get_tasks (
                 p_context            => 'SINGLE_TASK',
                 p_task_id            => :P1_TASK_ID,
                 p_show_expired_tasks => 'N' ) );
```

If a Human Task activity stores the task ID in a workflow variable, that ID can be used later in the workflow to read the task's state or parameters.

3.3 Completing a Task from Code

Tasks are normally completed by users, but they can also be completed programmatically, for example when a condition is met:

```
begin
  apex_human_task.complete_task (
    p_task_id => :TASK_ID,
    p_outcome => apex_human_task.c_task_outcome_approved );
end;
```

Always change tasks through the API rather than updating tables directly: the views are read-only, and the API keeps the workflow engine consistent.

3.4 Passing Data Between Tasks

Problem: Task 2 needs a value (for example a user name) that was entered in Task 1.

Approach:

1. In Task 1, store the value in a task parameter (for example by updating it from the Task Details page).
2. Store Task 1's ID in a workflow variable, e.g. `TASK1_ID`.
3. Between the two tasks, add an *Execute Code* or *Invoke API* activity that reads the parameter:

```
:TASK1_USER := apex_human_task.get_task_parameter_value (
                 p_task_id         => :TASK1_ID,
                 p_param_static_id => 'USER_NAME' );
```

4. Store the result in another workflow variable (here `TASK1_USER`).
5. In Task 2, use `TASK1_USER` in a parameter, in the subject, or to assign the task.

4 Workflows

4.1 How a Workflow Runs

A workflow follows its diagram. It moves from activity to activity according to task outcomes and workflow variables, running procedures, updating data and recording every step as it goes. Each human task it creates appears in the Unified Task List, and every user action (approve, reject, complete) is recorded by the workflow engine for auditing.

A workflow has a single *Start* activity and is started with a detail primary key. From then on, its details, transitions and states are logged and visible in the Workflow Console. Each activity stays active until it finishes: a human task until it is approved, rejected or completed, an automated activity until its code has run. The engine then moves to the next activity.

The `detail_pk` is propagated to every task and stored with it (see `APEX_TASKS`). It is what links all the data of one process together.

Example. In an expense-report process, the report ID groups all the expense lines submitted together. Passing that ID as the detail primary key lets every activity retrieve the full report (status, employee, lines, totals) without extra lookups.

```
-- details of one workflow instance
select *
  from table ( apex_workflow.get_workflows (
                 p_context     => 'SINGLE_WORKFLOW',
                 p_workflow_id => :P1_WORKFLOW_ID ) );

-- business records whose workflow was terminated
select detail_pk
  from apex_workflows
 where state_code = 'TERMINATED';
```

4.2 Workflow States

Workflow state transitions are made by the engine, through the API, or as the result of a user action; for example, a workflow moves from *Active* to *Completed* after a final approval.

State	Category	Possible transitions
Active	Normal	**Completed** when an end activity is reached. **Faulted** when the instance cannot be created (invalid parameters, no start activity or more than one) or an activity fails. **Suspended** by API or user request. **Terminated** by API or user request, or when an end activity set to terminate is reached.
Completed	Normal	None.
Faulted	Exception	**Active** when retried successfully. **Suspended** or **Terminated** by API or user request. A faulted workflow can be retried after its variables are fixed, or terminated if it should not run any more.
Suspended	Normal	**Active** when resumed. **Faulted** when an outstanding asynchronous activity returns an error. **Terminated** by API or user request. A workflow resumes at its current activity by default; `APEX_WORKFLOW.RESUME` can resume it at any activity (24.2).
Terminated	Exception	None. A terminated instance cannot be retried.

Table 3: Workflow states and transitions.

4.3 Activity States

An activity is always in exactly one state:

State	Category	Possible transitions
Active	Normal	**Completed** when it succeeds and the workflow continues. **Faulted** when it, or something leading into or out of it, fails. **Terminated** when the workflow is terminated. **Waiting** when it waits for a result, e.g. an approval or an asynchronous call.
Waiting	Normal	**Active** when the awaited operation finishes (this can be so quick that users never see the waiting state). **Faulted** if processing fails. **Terminated** if the workflow is terminated.
Completed	Normal	None.
Faulted	Exception	**Active** when the workflow is retried: the engine runs the activity again.
Terminated	Exception	None.

Table 4: Activity states and transitions.

4.4 Parameters and Variables

Workflow parameters are the inputs passed to a workflow when an instance starts; for a laptop-request approval, these could be the employee ID and the laptop type. They support data types and format masks and can be based on SQL queries or page items. Parameters have a direction:

- **In:** read-only inside the workflow.
- **Out** and **In/Out:** can be updated while the instance runs. They can only be based on workflow items (variables and parameters), and *Out* values can only be returned when the workflow is invoked from another workflow, not from a page process.

Workflow variables belong to a workflow version and can change at runtime. They support data types and format masks, can be initialised from SQL queries, expressions, function bodies or parameters, and may start as `NULL`. A variable is updated by:

- the result of a completed activity (for example a task outcome);

- a call to `apex_workflow.update_variables`;
- an administrator editing it in the workflow details page.

4.5 Development and Active Versions

A new workflow version starts in **development** mode, where it can be changed freely. Do all testing in this mode.

When the application is ready, the version must be **activated**. An active version can only be started when the application itself is not in development mode, and some of its features can no longer be edited or added.

Activating a new version makes the previous one obsolete, but instances already started keep running with the version they were created with. Workflow instances of an active version cannot be deleted: deactivate the version first, then remove its instances.

5 Troubleshooting

5.1 Error ORA-20987 When a Task Is Created

```
ORA-20987: APEX - The creation of a new Task for Task Definition <id>
in application <app_id> failed. SQLCODE: -20001
```

Likely causes:

- The task definition's SQL query (Action Source) is invalid or returns nothing. Fix it, or set the source to *None* if it is not needed.
- A task definition action that runs *on Create* raised an unhandled error. Check every action attached to the event at which the error appears.

5.2 Rolling Back Changes After a Workflow Is Terminated

When a workflow is terminated part-way, the changes its activities already made to the database stay in place. Two strategies help:

- **Revert on termination.** Run logic when a workflow is terminated that resets every value it changed. This works, but becomes slow and fragile if the workflow made many changes.
- **Write at the end.** Keep intermediate values in workflow variables and write them to the database only when the workflow completes. This is cleaner, but harder to manage in complex workflows.

Example. A workflow sets a record's status to "In review" when it starts, then creates a human task; when the task is completed, it sets the final status. If the workflow is terminated in between, an **automation** that looks for terminated workflows can set the status back and cancel the pending task:

```
select w.detail_pk
  from apex_workflows w
 where w.state_code = 'TERMINATED'
   and w.end_time   > apex_automation.get_last_run;
```

5.3 The Automation Does Not Run

- Check that the workflow's `state_code` really becomes `TERMINATED`.
- Temporarily remove the `end_time > apex_automation.get_last_run` filter and run it again.
- Check that `detail_pk` actually contains the expected record ID; if it is empty, the workflow was started without a detail primary key.
- Check that none of the columns used in the automation's conditions are `NULL`.

5.4 A User Cannot See a Task

- The user was created, or given the required role, after the task was created (see *Task Visibility Pitfalls*).
- Fix: retry the task, or have the initiator cancel it and start it again, so that potential owners are re-evaluated.
- To prevent it: assign roles before starting workflows, or add an automation that notifies the business administrator when a new user is added.

6 References

- Oracle, *Managing Workflows and Tasks* (APEX 24.1 App Builder guide): `https://docs.oracle.com/en/database/oracle/apex/24.1/htmdb/managing-workflows-and-tasks.html`
- Oracle, *APEX_HUMAN_TASK* package reference: `https://docs.oracle.com/en/database/oracle/apex/23.2/aeapi/APEX_HUMAN_TASK.html`
- Oracle, *APEX_WORKFLOW* package reference: `https://docs.oracle.com/en/database/oracle/apex/24.1/aeapi/APEX_WORKFLOW.html`
- Oracle, *Example workflows* (Sample Workflow, Approvals and Tasks app, available from the gallery under App Builder → Create): `https://docs.oracle.com/en/database/oracle/apex/24.1/htmdb/example-workflows.html`
- Oracle LiveLabs workshop on APEX workflows: `https://apexapps.oracle.com/pls/apex/r/dbpm/livelabs/run-workshop?p210_wid=3880`
- A. Chatterjee, tutorials on the Oracle APEX blog: `https://blogs.oracle.com/authors/ananya-chatterjee`
- P. Hartenfeller, *APEX 23.2 Workflows: first look*: `https://hartenfeller.dev/blog/apex-23-2-workflows-first-look`
- Novoshore, *Business processes with APEX 24.2 workflows*: `https://www.novoshore.com/2025/02/24/business-processes-with-apex-24-2-workflows-a-practical-demo/`
- *Handling terminations in Oracle APEX workflow*: `https://ebs-apex.hashnode.dev/handling-terminations-in-oracle-apex-workflow`
- Oracle APEX blog, *New in APEX 24.2: workflow within workflow, out parameters, copy and more*: `https://blogs.oracle.com/apex/post/new-in-apex-242-workflow-within-workflow-out-parameters-copy-and-more`

- Oracle APEX blog, *APEX 24.2: resume workflows from any activity*: `https://blogs.oracle.com/apex/post/apex-242-resume-workflows-from-any-activity`
- Oracle forums, *Updating workflow variable from task*: `https://forums.oracle.com/ords/apexds/post/updating-workflow-variable-from-task-6284`